

EGB240 Assessment 2: Digital Voice Record Design Report

Thomas Cox: n11024208

June 2024

Contents

1	Introduction	3
2	Project Description	3
3	Methodology	4
3.1	Anti-Aliasing Filter	4
3.2	Attempting a DC Removal/Buffer	6
3.3	Amplification	6
3.4	Circuit Improvements	7
4	Theoretical and Experimental Data	8
4.1	Theoretical	9
4.2	Experimental Data	11
5	Future Work	13
6	Appendix	14
6.1	A	14

List of Figures

1	DVR Block Diagram	3
2	The First Iteration	4
3	Ideal Bode Plot	5
4	First DC Buffer iteration	6
5	First Amp Iteration	7
6	Initial Combined Circuit	7
7	Reference voltage implementation	8
8	Theoretical DC Buffer	9
9	Theoretical Bode Plot Obtained from LTSpice	10
10	Theoretical voltage output	10
11	Assembled prototype	11
12	Experimental DC Buffer Output	12
13	width =	13

List of Tables

1	Pushbutton Functionality	4
2	Key Matlab Values	5

1 Introduction

Using a given microcontroller, we were tasked with prototyping the input signal conditioning for a digital voice recording (DVR) (fig.1). Following this we are to test and capture both theoretical results and experimental results using LTSpice and a digital oscilloscope respectively.

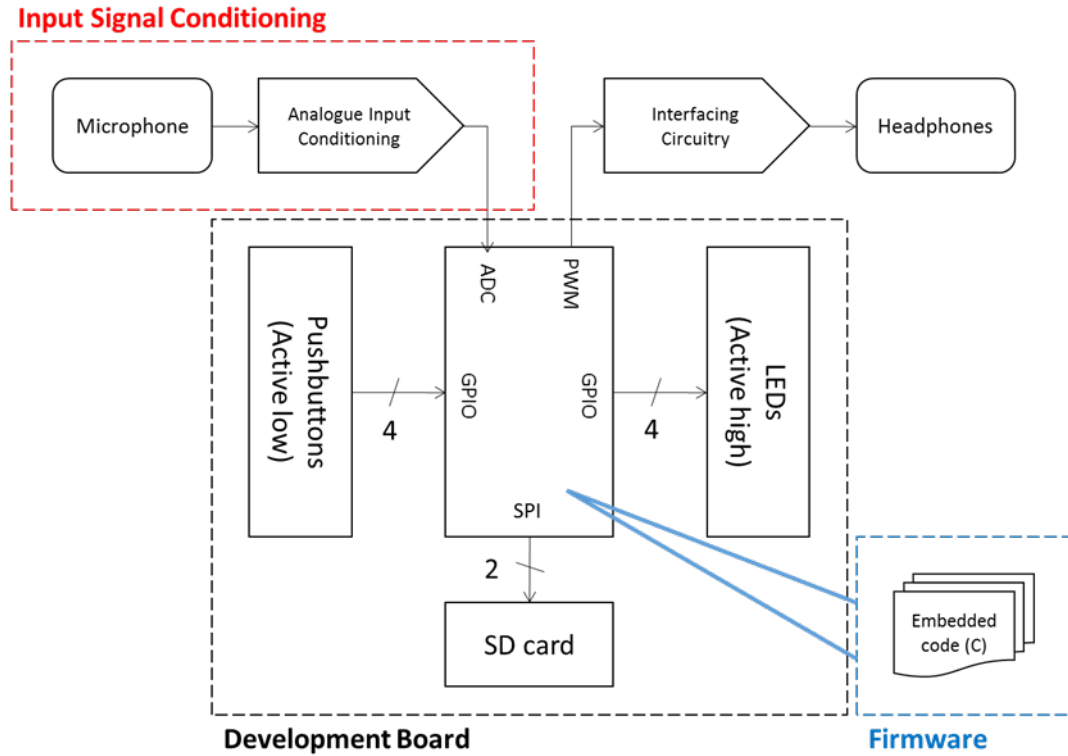


Figure 1: DVR Block Diagram

2 Project Description

As stated previously, the DVR must be compatible with a microcontroller. This microcontroller is a custom board designed by the Queensland University of Technology (QUT) and contains a Teensy 2.0 board, an SD memory card slot, pushbutton functionality (table 1), and LED output indicators.

This microcontroller is powered by a 5V USB. For prototyping, a lab power supply set to 5V and 0.03A was used.

Pushbutton 1	Play
Pushbutton 2	Record
Pushbutton 3	Stop

Table 1: Pushbutton Functionality

To successfully implement the input signal conditioning, four key parts were necessary.

1. Microphone connection
2. DC Removal and Buffer/Amplifier
3. Anti-aliasing Filter
4. Amplifier

With these four items, the signal from the microphone would successfully interface with the microcontroller.

3 Methodology

3.1 Anti-Aliasing Filter

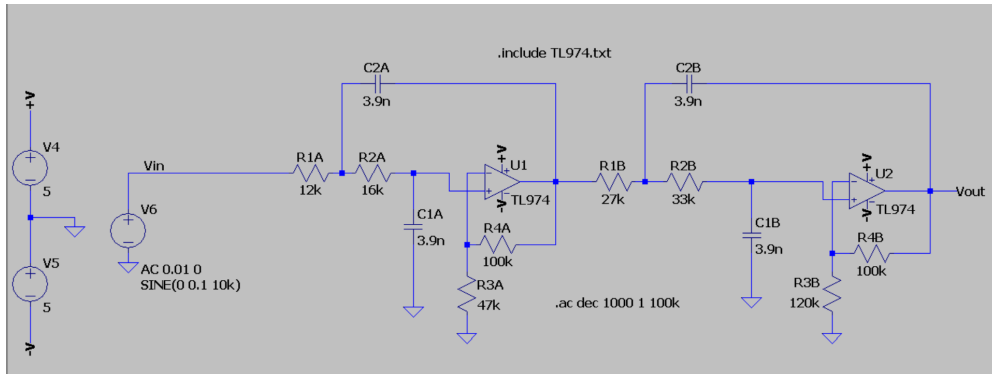


Figure 2: The First Iteration

Over the course of several weeks, multiple iterations were tested. As shown above, this is the first attempt. This circuit was primarily focused on applying an anti-aliasing filter. Due to its sharp cutoff, the Chebyshev filter was chosen. To determine filter characteristics, a Matlab script (Appendix A) was run, which then outputs several key components.

Variable	Value
f_pass	2800 Hz
f_s	7.9 kHz
order	4

Table 2: Key Matlab Values

Additionally, with this code, an ideal bode plot was provided.

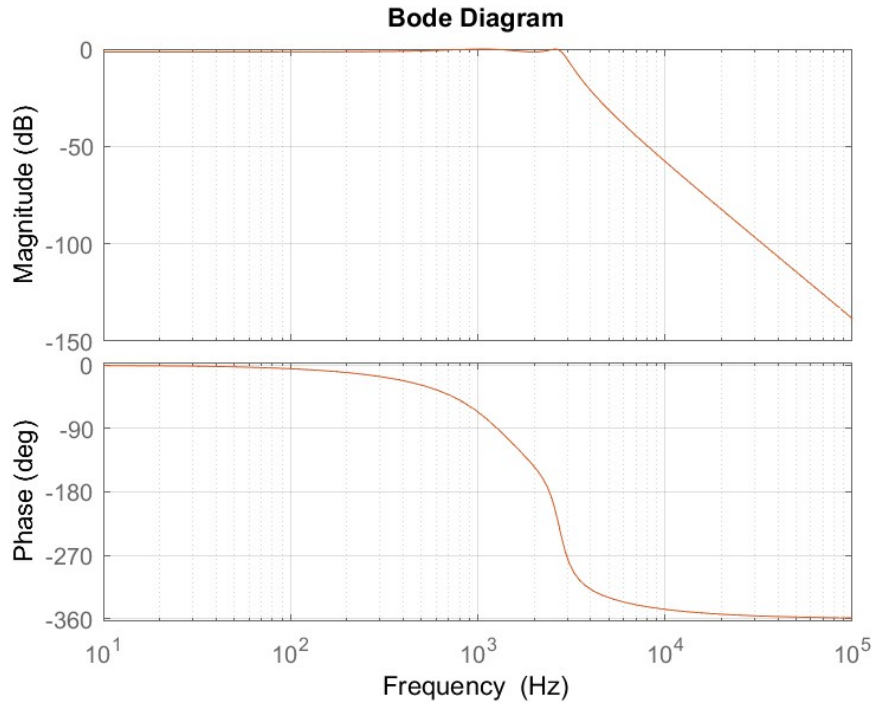


Figure 3: Ideal Bode Plot

Additionally, with this code, an ideal bode plot was provided. With this information, I then used the following equation to help select component values:

$$\frac{V_o(s)}{V_{in}(s)} = \frac{\frac{K}{R_1 R_2 C_1 C_2}}{s^2 + s\left(\frac{1}{R_1 C_2} + \frac{1}{R_2 C_2} + \frac{1-k}{R_2 C_1}\right) + \frac{1}{R_1 R_2 C_1 C_2}} \quad (1)$$

$$K = 1 + \frac{R_4}{R_3} \quad (2)$$

This information is what was used in the circuit shown in Fig. 2.

3.2 Attempting a DC Removal/Buffer

In order to start getting usable data, the next thing to be added was the microphone connection and the DC removal. This circuit would isolate the input signal, preventing any dc offset or bias from affecting the following parts of the circuit and ensuring that the input signal is clean and free from dc voltage shifts that could distort a signal from the microphone.

With this goal, a simple buffer was tested:

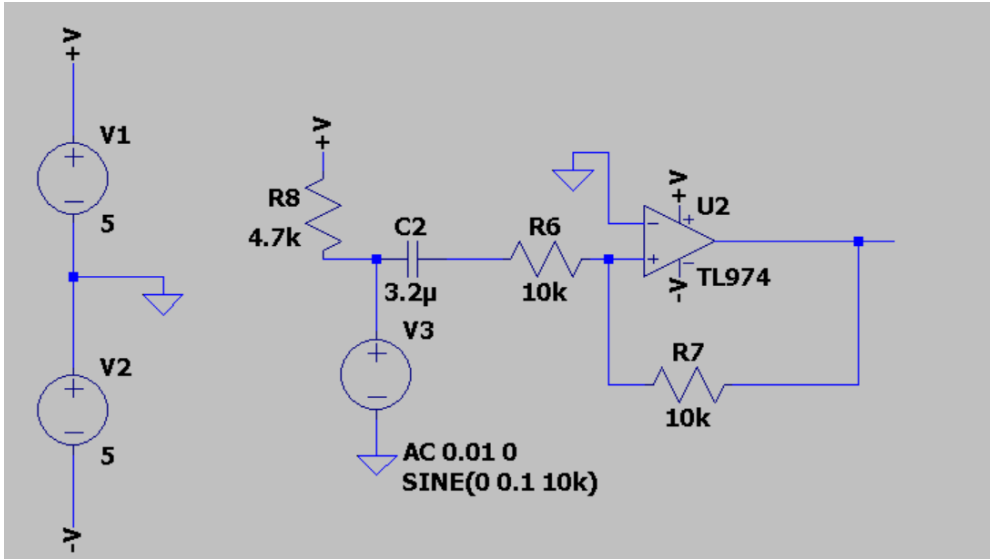


Figure 4: First DC Buffer iteration

Notice that this buffer also contains the microphone connection circuit, which consists of a load resistor and capacitor, R8 and C2, respectively.

3.3 Amplification

When creating the amplifier, a waveform generator was used to mimic human speech to get appropriate data. This wave form was a 100mV 120Hz sin wave. This frequency is within the starting range of what the average adult male speaks[1].

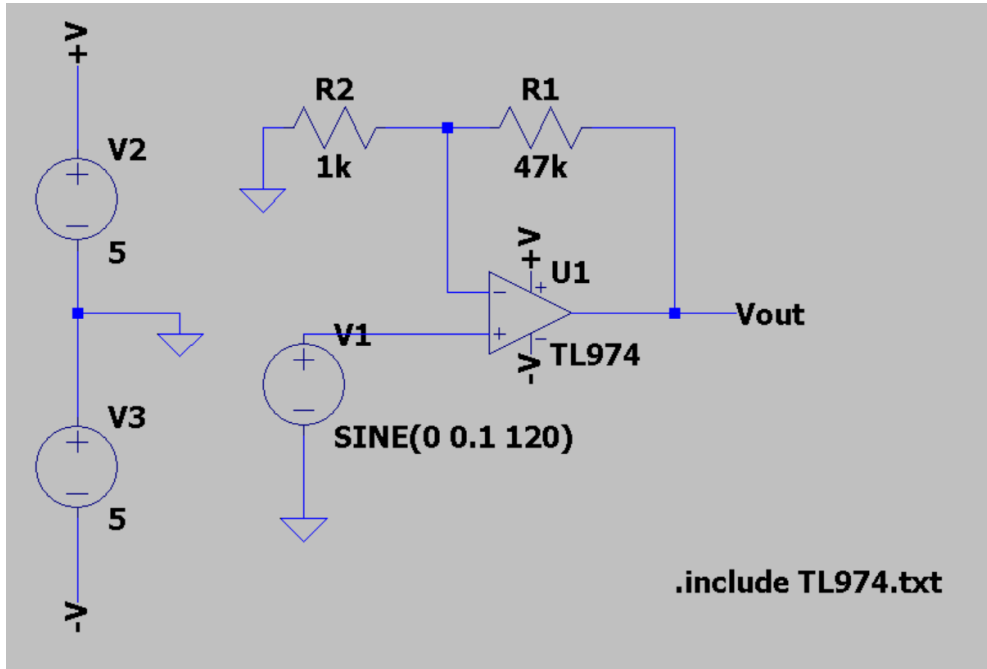


Figure 5: First Amp Iteration

Figure 5 shows the first attempt at an active lowpass filter. Although not completely functional, it would be improved quickly.

3.4 Circuit Improvements

With each individual component having a starting place, they were then all put into the one continuous circuit shown below.

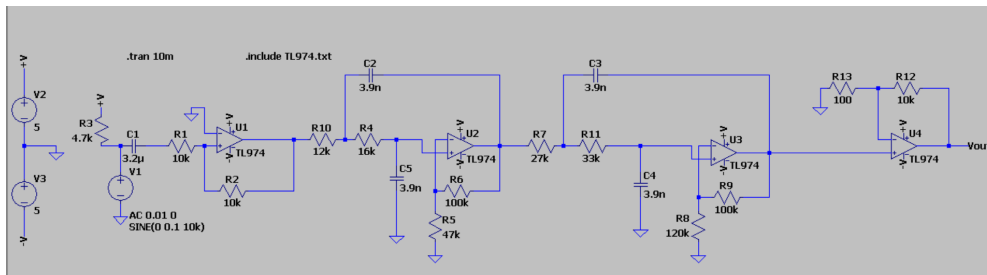


Figure 6: Initial Combined Circuit

After combining these, two large mistakes were quickly noticed.

1. The circuit had the wrong order of inverting and non-inverting amplifiers
2. There was no reference voltage for the dividers to go off of

This was quickly rectified with a reference voltage of 2.5 volts.

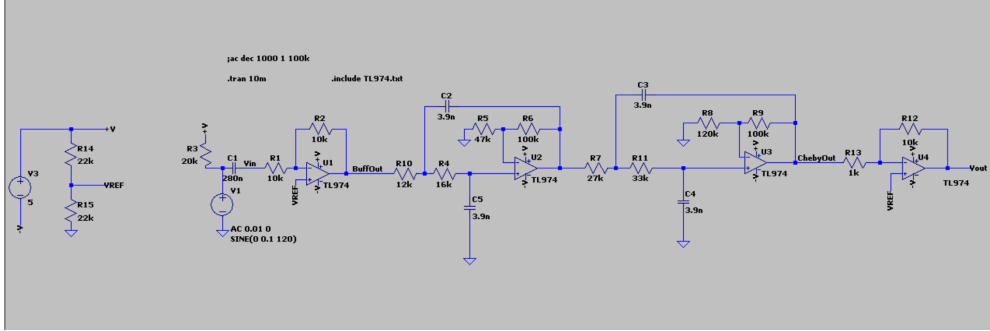


Figure 7: Reference voltage implementation

It is important to note that a couple of component values were changed in this step as well, but they resulted in the same output.

4 Theoretical and Experimental Data

Figure 7 displays my final iteration of the circuit that would then be used to obtain data. The data that would be collected would be taken from three key net labels. "BuffOut," "ChebyOut," and "Vout."

4.1 Theoretical

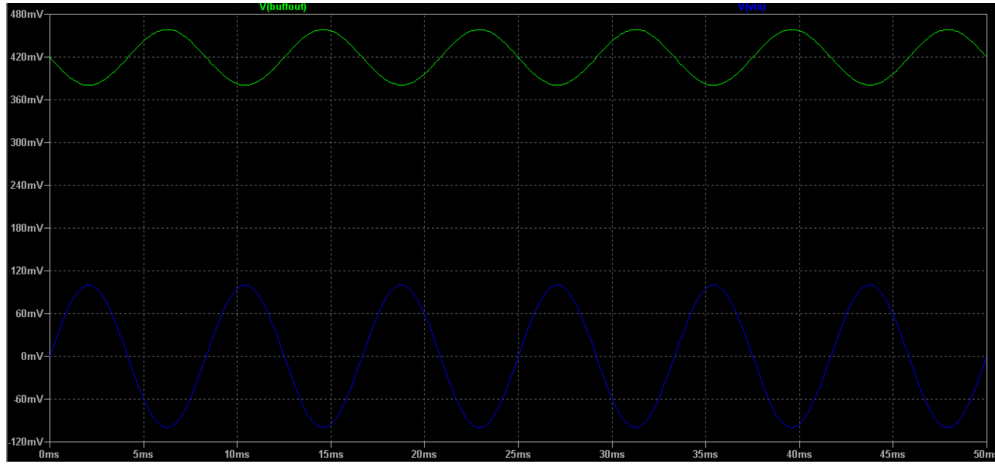


Figure 8: Theoretical DC Buffer

Shown here is the theoretical data obtained from the previously shown DC buffer. This figure indicates that the implemented DC buffer should successfully prevent any outside noise.

In Figure 3, we mentioned that this would be the ideal Bode plot that was generated by Matlab. When using LTSpice to obtain a bode plot from these parameters, the following was obtained:

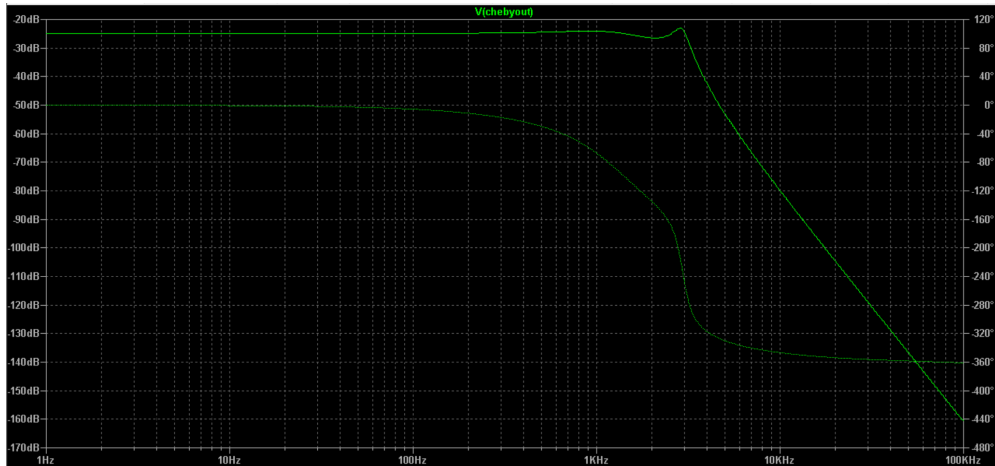


Figure 9: Theoretical Bode Plot Obtained from LTSpice

The final net label, Vout, was then probed to make sure the output voltage levels would be suitable.

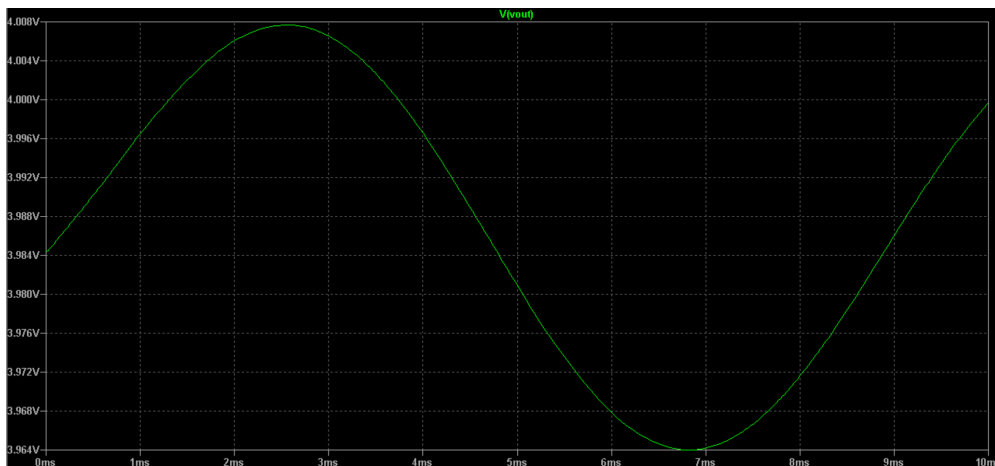


Figure 10: Theoretical voltage output

This graph shows that the output level should not reach more than 4 volts, which gives plenty of room for error with a 5V power supply.

4.2 Experimental Data

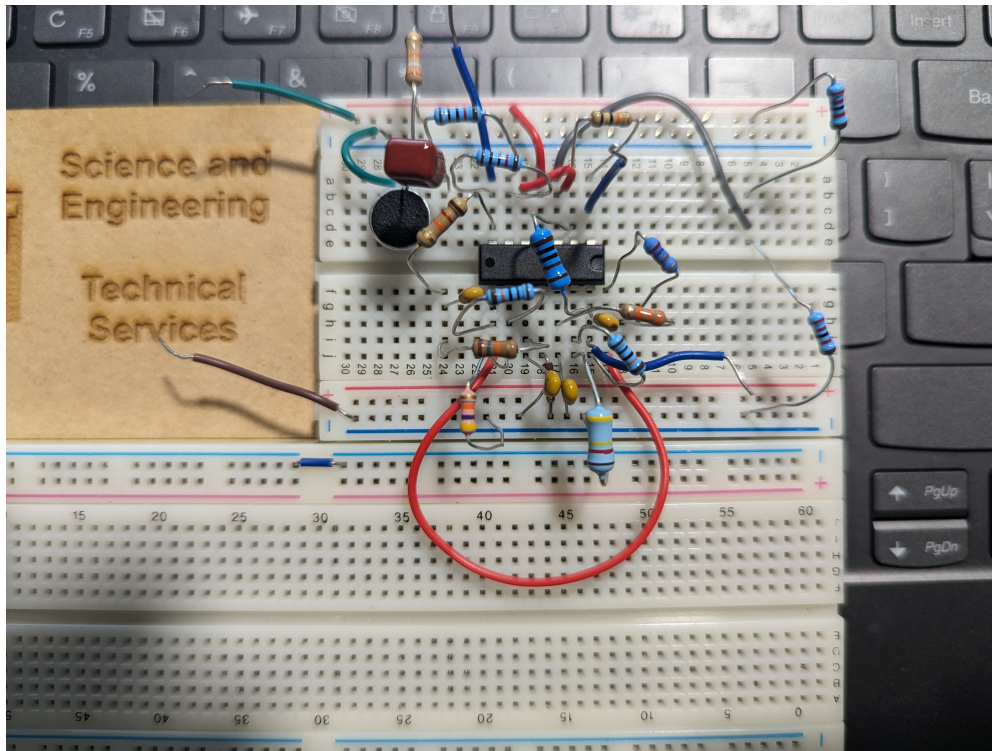


Figure 11: Assembled prototype

The above image displays the assembled prototype, with each blue wire showing the probing points used. With this, the following data was obtained:

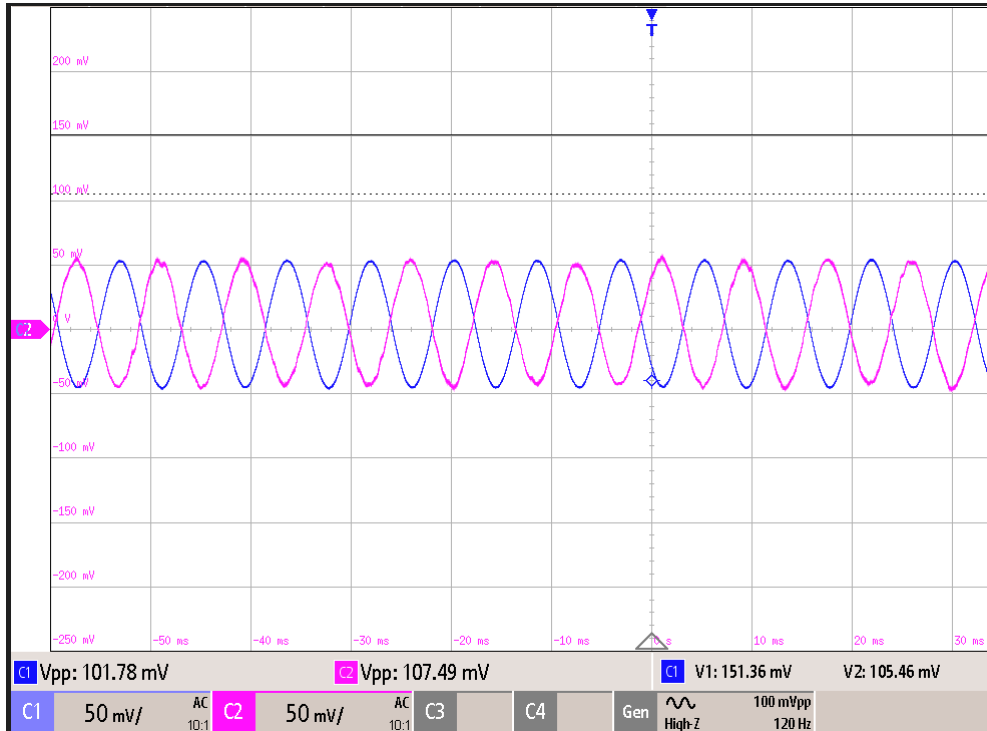


Figure 12: Experimental DC Buffer Output

This data was obtained at what would be a net-label BuffOut. It demonstrates that the implemented DC buffer is in fact acting accordingly and will prevent any noise. After obtaining this, a bode plot was obtained using the oscilloscopes in the built-in bode function.

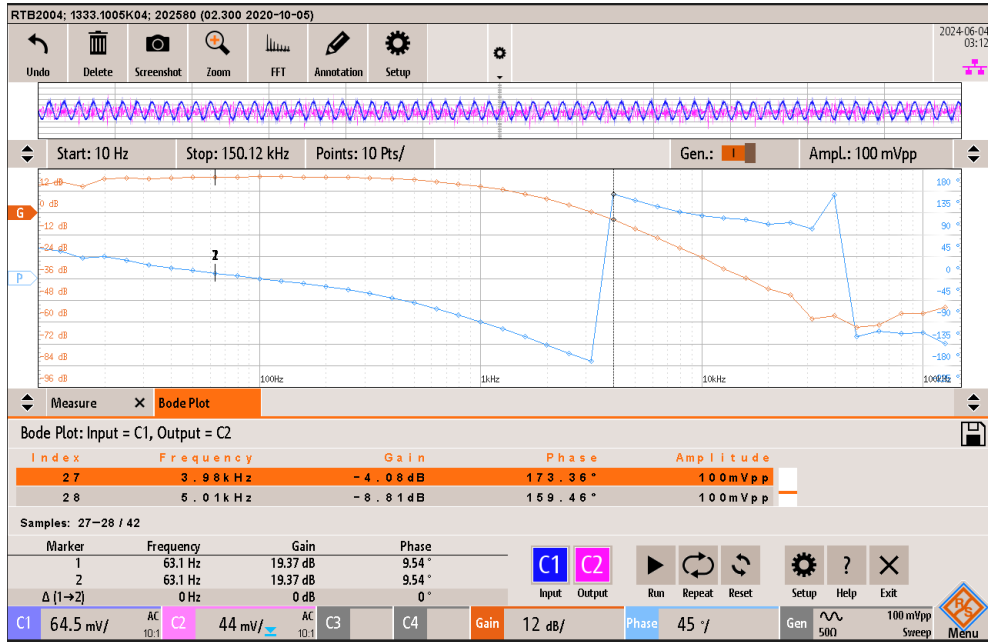


Figure 13: Experimental Bode plot

While not entirely ideal, this bode plot shows that the filter is attenuating at the same frequency as the theoretical data, which is around 3 kHz. The phase, however, showed the correct waveform until just prior to 4 kHz. This could be due to someone bumping the probe, faulty components, or other external factors.

On the top part of the image, you can see the input and output waveforms that were used during the bode plot. It can be seen that the output (blue line) was consistently peaking at around 3 volts. While this is less than the theoretical 4V, the only impact it should have on performance is a slightly quieter, but still audible, output audio.

5 Future Work

Overall, the results from this project were conclusive and desirable, with the exception of one variable. The point at which the filter would attenuate, f_s. The theoretical output shown by Matlab was that it would attenuate at around 7.9 kHz, but the experimental results output an attenuation of around 3 kHz. While this does still cover the majority of the average speaking frequencies [1] it would be largely beneficial for future iterations to change the capacitor values in the Chebyshev filter to obtain this ideal attenuation factor.

References

- [1] R. Quam, I. Martínez, C. Lorenzo, *et al.*, “Studying audition in fossil hominins: A new approach to the evolution of language?” In *Psychology of Language*, Journal Abbreviation: Psychology of Language, Oct. 2012, pp. 1–37, ISBN: 978-1-61942-819-5.

6 Appendix

6.1 A

```
%%% Filter specs
fsamp = 15.625e3;

fs = fsamp/2;      % stopband frequency, Hz
fp = 2.8e3;        % passband frequency, Hz

ws = 2*pi*fs;
wp = 2*pi*fp;

Amin = 20*log10(1/2^8); % stopband attenuation, dB    48dB
Amax = 1.5;           % passband attenuation, dB

%%% Calculate required order of Chebyshev filter
[n_cheb, wn_cheb] = cheb1ord(wp, ws, Amax, Amin, 's');

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
[b, a] = cheby1(n_cheb, Amax, wn_cheb, 'low', 's');
G = tf(b, a);

figure(1);
h = bodeplot(G); %
    display bode plot
setoptions(h, 'FreqUnits', 'Hz'); %
    change units to Hz
grid on;
hold;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%% Stage A %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
[z, p, k] = tf2zpk(b, a);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%% Stage A %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
a1A = -p(1) - p(2);
a2A = p(1) * p(2);
```

```

wnA = sqrt(a2A);
QA = wnA / abs(a1A);

fprintf('Stage A: wn = %g rad/s, Q = %g\n', wnA, QA);

%%%% Stage B %%%%%%%%%%%%%%
a1B = -p(3) - p(4);
a2B = p(3) * p(4);

wnB = sqrt(a2B);
QB = wnB / abs(a1B);

fprintf('Stage B: wn = %g rad/s, Q = %g\n', wnB, QB);

```